Live Speaker Diarization — Complete Project Guide

Real-time “who is speaking” on live streams · from-scratch explanation · open-source · GPU or CPU · Windows / Linux / macOS

1 · WHAT THIS PROJECT IS

Speaker diarization answers **“who spoke when?”** — it splits audio into time segments and labels each with a speaker (`SPEAKER_00`, `SPEAKER_01`, …), giving the *same voice* the *same label* throughout. It does **not** name people; it distinguishes and tracks *distinct voices*. This project does it **live**: on a never-ending stream, it reports the current speaker within a few seconds, continuously.

Two settings of the same problem

- **Offline** (foundation): the whole recording is available at once → highest accuracy.
- **Online / real-time** (this repo): audio arrives forever; you must decide *now*, with only recent context, and never fall behind.

Two classic failures we defeat

- **Fragmentation:** one real person split into many phantom speaker IDs.
- **Label permutation:** speaker labels randomly swapping between chunks.

Solved by a neural diarizer + a **persistent speaker registry** (below).

2 · THE NEURAL DIARIZATION FOUNDATION (OFFLINE CORE)

Both offline and online build on the same neural pipeline. Audio is decoded to **16 kHz mono**, then processed in five stages:

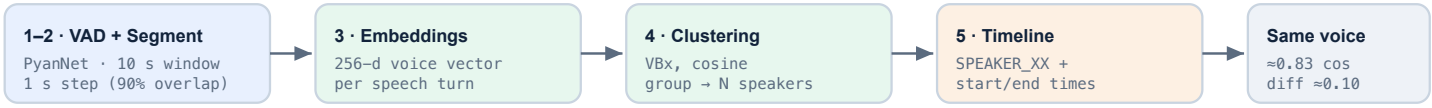

Fig. 1 — The neural pipeline. Steps 1–3 are neural nets; step 4 clusters the voice fingerprints. The wide 0.83-vs-0.10 gap makes speakers separable.

Why it doesn’t fragment

Clustering sees many turns with a **broad view**, so an expressive speaker’s varied turns still group together. Two safeguards clean the edges: an adjustable **clustering threshold**, and merging of tiny low-evidence clusters into the nearest real voice.

Known vs unknown speaker count

Telling the model the exact count (`--speakers N`) is most reliable. Otherwise it estimates the count — convenient but never perfect for every recording (an inherent limitation).

3 · GOING LIVE — THE REAL-TIME ARCHITECTURE

A live stream never ends, so you can't re-process the past. Instead we keep a short **rolling buffer** of recent audio, re-diarize it on a fixed cadence, and use a **persistent speaker registry** to keep IDs stable. Ingestion and compute are **decoupled by a thread** so latency stays bounded.

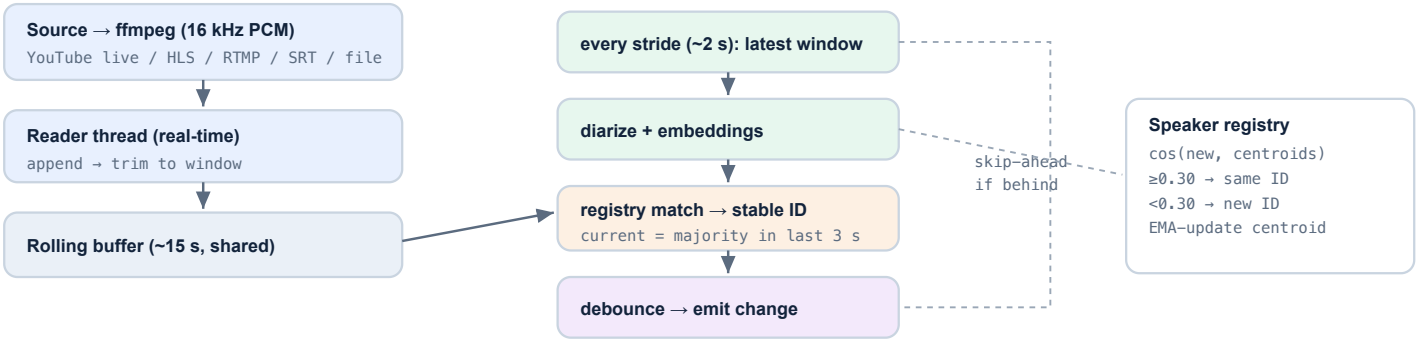

Fig. 2 — Real-time engine. The reader thread never blocks on compute; the main loop always diarizes the newest window and skips ahead if slow, so latency is bounded.

The speaker registry (the heart)

Each re-diarization invents fresh local labels. The registry pins identity to the **voice fingerprint**: a new cluster is matched to the most-similar stored centroid (cosine ≥ `0.30`) and keeps that stable ID; otherwise it becomes a new speaker. Centroids update with an EMA, absorbing natural variation. This is what defeats label permutation live.

Bounded-latency scheduling

On an Apple M1 GPU a 15 s window re-diarizes in ~**1.4 s** — well under the 2 s stride, so it keeps pace. If a stride ever runs long, the loop simply diarizes less often instead of piling up a backlog. A short **debounce** (same speaker for ≥2 strides) suppresses spurious flips.

4 · INGESTION — GETTING LIVE AUDIO IN

Source	How
Direct stream (HLS `.m3u8` / RTMP / SRT / internet radio)	fed straight to `ffmpeg` → 16 kHz mono PCM. No scraping. This is the production path (e.g. a broadcast encoder feed).
Local file (`--simulate`)	replayed at 1× real-time through the identical pipeline — the best first test, fully offline.
Live YouTube URL	`yt-dlp` resolves the live HLS manifest; YouTube's anti-bot JS challenge requires browser cookies + a JS runtime + yt-dlp's solver (advanced, opt-in).

5 · ALL PARAMETERS & DEFAULTS

Parameter	Default	Role	What it controls
Sample rate	16 kHz mono	input	required audio format
`segmentation window`	10.0 s	pyannote internal	the neural analysis “chunk”
`segmentation step`	1.0 s	pyannote internal	hop between chunks (90% overlap)
`embedding dim`	256	pyannote internal	voice fingerprint size
`window`	15 s	**live knob**	rolling buffer length (context per pass)
`stride`	2 s	**live knob**	re-diarize cadence (latency vs compute)
`clustering threshold`	0.85	**tuning knob**	merge aggressiveness (over-split vs merge)
`num_speakers`	*auto*	**tuning knob**	set a known count for best accuracy
`registry match_sim`	0.30	registry	same-voice cosine cutoff (same≈0.83, diff≈0.10)
`warmup / recent / confirm`	8 s / 3 s / 2	live	startup gate · recency for “current” · anti-flip debounce

6 · RUNNING IT

```
python live_diarize.py sample.wav --simulate                # simulated live (no network)
python live_diarize.py "https://.../live.mp3" --max-seconds 60   # direct stream
python live_diarize.py "https://youtube.com/watch?v=ID" \       # live YouTube (advanced)
    --cookies-from-browser chrome --js-runtime deno --remote-components ejs:npm
```

Live output (any mode):

```
[wall   10.0s | stream    9.5s]  * SPEAKER 1  [NEW VOICE]
[wall   40.1s | stream   39.5s]  * SPEAKER 2
  speakers seen: 2 [1, 2]   changes: 2
```

7 · PERFORMANCE (MEASURED)

Real-time

~1.4 s compute per 15 s window on a laptop GPU (Apple M1); keeps pace with the live edge at a 2 s stride.

Accurate

Online result ≈ **96%** frame-agreement vs the offline (whole-file) gold — no fragmentation.

Portable

Auto-uses NVIDIA CUDA, Apple MPS, or CPU. GPU strongly recommended for real-time.

8 · REQUIREMENTS & HONEST LIMITATIONS

- **GPU:** NVIDIA CUDA (or Apple MPS) for real-time; CPU can’t keep up. Install a **matched** torch + torchaudio pair (same version, same CUDA index).
- **Model access:** free Hugging Face token + accept the `pyannote/speaker-diarization-community-1` terms once; weights cache locally.
- **Latency:** a few seconds after a real change (buffer fill + confirmation) — tunable via `stride` / `window`.
- **Unknown count:** no single threshold is perfect for every stream — prefer `--speakers N` when known.
- **Overlap:** when people talk at once, each instant is attributed to one speaker.
- **YouTube live:** depends on yt-dlp getting past YouTube’s anti-bot challenge; direct stream feeds avoid this entirely.

Stack: `pyannote-audio 4` · `community-1` · wespeaker 256-d embeddings · `VBx` clustering · `PyTorch` (CUDA/MPS) · `ffmpeg` · `yt-dlp` — all open-source, no paid APIs, no cloud.